

Traxa — شرح الأكواد

دليل مبسط للمبتدئين في Flutter

شرح فانكشن فانكشن، Container Container
من زاوية مبتدئ Flutter

Generated: 2026-05-29

0. مفاهيم Flutter التي تحتاج تفهمها قبل القراءة

0.1 ما هو ال Widget؟

كل حاجة تشوفها على الشاشة في Flutter هي **Widget**: زر، نص، صورة، حتى المساحة الفارغة (Padding/SizeBox). ال Widgets بتتركب في شجرة (Widget Tree) — الأب يحتوي على أولاد، الأولاد لهم أولاد، وهكذا.

```
Scaffold          ← Widget ال App الأساسي (يعمل layout ال AppBar + Body + BottomNav)
└─ Column          ← يرتب أولاده عمودياً
    └─ Text(...)    ← نص
    └─ SizedBox(h:8) ← مسافة فاضية ارتفاعها 8
    └─ ElevatedButton ← زر
```

StatelessWidget vs StatefulWidget 0.2

Stateless	Stateful
ما عندوش بيانات تتغير. ترسم نفس ال UI كل مرة.	عنده State ممكن يتغير → ال Widget يعيد البناء.
أمثلة: شعار، عنوان ثابت، Container زخرفة.	أمثلة: Form، Counter، شاشة بتحمل بيانات من API.

ال StatefulWidget لما `setState()` ينادى، Flutter يعيد بناء ال widget بالبيانات الجديدة.

BuildContext ال 0.3

ال **context** هو المرجع لمكان ال Widget في الشجرة. ممكن منه نوصل لل Theme، لل MediaQuery (المقاسات)، أو لأي **Cubit/Provider** فوقه عبر `context.read<X>()` أو `context.watch<X>()`.

Container 0.4 — أهم Widget هنشوفه كتير

Container هو "صندوق" بيتلف widget. بيقدّر يضيف:

- **padding** — مسافة داخلية حول المحتوى.
- **margin** — مسافة خارجية حوالية.
- **decoration** — لون، `gradient`، `border`، `borderRadius`، `shadow`.
- **width / height** — مقاسات.
- **alignment** — مكان المحتوى جواه.

State Management — BLoC / Cubit ال 0.5

المشكلة: لو ال data في شاشة 1 محتاجين نعرضها في شاشة 5، إزاي ننقلها؟ الحل التقليدي (تمرير constructor parameters) صعب لما الشجرة كبيرة.

الحل: Cubit (نسخة مبسطة من BLoC). ال Cubit class عنده **state**، ال widgets تستمع له (**BlocBuilder**) وتعيد البناء عند تغييره. أي widget يقدر ينادي method على ال Cubit عبر `context.read<MyCubit>` `.doSomething()`.

```
class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0); // الحالة الابتدائية ←
  void increment() => emit(state + 1); // emit جديدة state بيرسل
}

// في UI:
```

```
BlocBuilder<CounterCubit, int>(
  builder: (context, count) => Text('$count'),
);
```

Future و async / await 0.6

أي حاجة بتأخذ وقت (HTTP request، قراءة ملف) هي **Future**. كلمة **await** تستنى لحد ما تخلص. الدالة اللي فيها await لازم تبقى **async**.

```
Future<String> getName() async {
  await Future.delayed(Duration(seconds: 2)); // استنى ثانيتين
  return 'Ali';
}
// استخدام: final name = await getName();
```

ValueNotifier 0.7

متغير بسيط بقيمة واحدة، أي حد بيستمع له يتنبه لما تتغير. أخف من Cubit للحالات الـ ultra-local (مثلاً: هل الـ password مخفي ولا لأ؟).

1. main.dart — البداية

1.1 main () — أول دالة تشتغل

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await ServerConfig.load();
  HttpOverrides.global = MyHttpOverrides();
  await initServiceLocator();
  runApp(const MyApp());
}
```

سطر سطر:

1. Flutter — `WidgetsFlutterBinding.ensureInitialized()` يستخدم محرك ++C تحت السطح. ال binding هو الجسر بين Dart والمحرك. لازم نفعله أول حاجة لو هنعمل async work قبل `runApp()` (زي قراءة من `SharedPreferences`).
2. `await ServerConfig.load()` — يقرأ URL السيرفر اللي المستخدم حفظه آخر مرة من `SharedPreferences`. (مثلاً `https://192.168.1.17` لو شغال محلي).
3. `HttpOverrides.global = MyHttpOverrides()` — يجبر التطبيق إنه يثق بـ SSL shadhada للسيرفر اللي محدد (مفيد لل self-signed certs).
4. `await initServiceLocator()` — يسجل كل ال services وال cubits في GetIt (ال Container ال DI). شوف القسم اللي جاي.
5. `runApp(MyApp())` — يبدأ تشغيل ال Flutter app ويرسم MyApp على الشاشة.

1.2 MyHttpOverrides

```
class MyHttpOverrides extends HttpOverrides {
  @override
  HttpClient createHttpClient(SecurityContext? context) {
    return super.createHttpClient(context)
      ..badCertificateCallback = (cert, host, port) {
        return host == Uri.parse(AppConstants.baseUrl).host;
      };
  }
}
```

```
}
}
```

ليه موجود؟ لو السيرفر شغال على HTTPS بشهادة self-signed (مثلاً سيرفر تطوير محلي)، Flutter بيرفضها افتراضياً. الكود ده بيقول "تجاهل الخطأ لو ال host هو السيرفر بتاعنا فقط". في ال production السيرفر يستخدم شهادة حقيقية فالكود ده مش بيؤثر.

ال **..** (**cascade operator**): طريقة لكتابة multiple statements على نفس ال object بدون تكرار

اسمه. `obj.a()..b() = obj.a(); obj.b()`

MyApp 1.3

```
class MyApp extends StatelessWidget {
  Widget build(BuildContext context) {
    return AppProviders(
      child: BlocBuilder<ThemeCubit, ThemeState>(
        builder: (context, themeState) {
          return ScreenUtilInit(
            designSize: const Size(393, 873),
            builder: (context, child) => MaterialApp(
              themeMode: themeState.themeMode,
              theme: AppTheme.lightTheme,
              darkTheme: AppTheme.darkTheme,
              home: child,
            ),
            child: const _AuthGate(),
          );
        },
      ),
    );
  }
}
```

1. **AppProviders** — يلف التطبيق ويعطيه الوصول لكل ال Cubits (Auth, Data, Theme).
2. **BlocBuilder<ThemeCubit>** — يعيد البناء كل ما ال theme يتغير.
3. **ScreenUtilInit** — يفعل ال system ال responsive design (يخلي `w/.h/.sp` تشتغل). `designSize` هو مقاس 393×873 Pixel — كل المقاسات بتتسبب نسبة له.
4. **MaterialApp** — جذر التطبيق. بيحدد ال theme والشاشة الأولى (home).

.AuthGate_ = home: child — ال child هنا

AuthGate_ 1.4 — البواب اللي بيقرر فين تروح

```
class _AuthGate extends StatelessWidget {
  Widget build(BuildContext context) {
    return BlocConsumer<AuthCubit, AuthState>(
      listenWhen: (prev, curr) => curr.isAuthenticated && curr.user != null &&
        (prev.user?.id != curr.user?.id || !prev.isAuthenticated),
      listener: (context, authState) {
        context.read<DataCubit>().fullReload();
      },
      builder: (context, authState) {
        if (authState.isAuthenticated && authState.user != null) {
          if (authState.user!.isDoctor || authState.user!.isTeachingAssistant) {
            return const DoctorScreen();
          }
          return const StudentScreen();
        }
        return const LoginScreen();
      },
    );
  }
}
```

- **BlocConsumer** = BlocBuilder + BlocListener معاً.
- **listener**: يشغل side-effect (مش بيغير ال UI). هنا: لما المستخدم يسجل دخول ناجح → DataCubit يعمل reload للبيانات.
- **builder**: يعيد بناء ال UI حسب الحالة.
- **listenWhen condition**: نشغل ال listener فقط لما يكون فعلاً متعمل login جديد (مش كل rebuild).
- **builder logic**: لو فيه user → روح لشاشته. لو لآ → LoginScreen.

2. ال Cubits — قلب State Management

AuthCubit 2.1 — تسجيل الدخول والخروج

الحالة (AuthState)

```
class AuthState extends Equatable {
  final User? user;
  final String? token;
  final bool isAuthenticated;
  final LoadingState loadingState;
}
```

Equatable = مكتبة بتعمل `==` تلقائي. ده مهم لأن BLoC ما بيني ال UI تاني إلا لو ال state اتغير فعلاً.

دوال AuthCubit

()init() / checkAuth

```
Future<void> checkAuth() async {
  final result = await _repo.restoreSession();
  result.when(
    success: (session) async {
      if (session == null) {
        emit(AuthState.initial());
        return;
      }
      emit(AuthState.success(session.user, session.token));
      await WebSocketService.instance.connect();
    },
    failure: (e) {
      emit(AuthState.error('Failed to restore session'));
    },
  );
}
```

الخطوات:

1. نسأل ال Repository: "في session محفوظة؟"

2. `result.when(success, failure)` — تتعامل مع الحالتين.
3. لو `session = null` (مفیش حد سجل من قبل): emit initial state (شاشة Login).
4. لو موجودة: emit success → ال AuthGate يفتح الشاشة المناسبة + نشغل WebSocket.

لماذا restore session؟ لما المستخدم يقفل التطبيق ويفتحة تاني، عايز يلاقي نفسه مسجل دخول. السيشن محفوظة في SecureStorage (مشفرة).

()login

```
Future<void> login({required username, password, required isStudent}) async {
  emit(AuthState.loading());
  final result = isStudent
    ? await _repo.studentLogin(...)
    : await _repo.doctorLogin(...);
  result.when(
    success: (session) async {
      emit(AuthState.success(session.user, session.token));
      await WebSocketService.instance.connect();
    },
    failure: (e) => emit(AuthState.error(e.message)),
  );
}
```

الخطوات:

1. emit loading → شاشة Login تعرض loading indicator على الزرار.
2. حسب التوجل (طالب/دكتور)، نستدعي ال method المناسبة.
3. لو نجح: نخزن user + token + نشغل WS.
4. لو فشل: نعرض رسالة الخطأ.

()logout

```
Future<void> logout() async {
  await _repo.logout();
  WebSocketService.instance.disconnect();
  emit(AuthState.loggedOut());
}
```


ينظف كل حاجة: SecureStorage، token in-memory، ال WebSocket. ثم AuthGate → emit loggedOut يرجع للـ LoginScreen.

DataCubit 2.2 — البيانات الأكاديمية

الحالة (DataState)

عندها قوائم لكل حاجة: students, doctors, subjects, lectures, sections, grades, attendance, .teachingAssistants + currentSemester + currentAcademicYear

أهم دالة: loadAllData()

```
Future<void> _fetchAndEmit({required String errorPrefix}) async {
  emit(state.copyWith(loadingState: LoadingState.loading()));

  await loadCurrentSemester();
  await loadCurrentAcademicYear();

  final results = await Future.wait([
    _studentRepo.getAllStudents(),
    _doctorRepo.getAllDoctors(),
    _doctorRepo.getSubjects(),
    _doctorRepo.getLectures(),
    _system.getSections(),
    _taRepo.getAll(),
  ]);
  ...
}
```

الخطوات:

1. emit loading state.
2. نجيب السمسטר والسنة الأكاديمية الحالية.
3. **Future.wait** — يشغل الـ 6 calls بالتوازي (في نفس الوقت) بدل التتابع. أسرع 6 مرات تقريباً!
4. نفحص هل في حاجة فشلت.
5. لو فشلت كلها → نحاول cache.
6. لو نجحت → نحول الـ JSON لـ Models و emit الحالة الجديدة + نخزن في cache.

emitLoaded_ — تحويل JSON إلى Models

```
final allStudents = rawStudents.map((j) => Student.fromJson(j)).toList();
final allDoctors = rawDoctors.map((j) => Doctor.fromJson(j)).toList();
final allSubjects = rawSubjects.map((j) => Subject.fromJson(j)).toList();
...

final filteredSubjects =
  allSubjects.where((s) => s.semester == _currentSemester).toList();
```

التحويل: `map` يتحوّل كل عنصر في القائمة لـ `object Dart` نظيف. ثم `toList()` ترجع `List` حقيقية.
الفلتر: `where` بترجع العناصر اللي تطابق الشرط. هنا نخلي `subjects` الترم الحالي فقط.

Dark/Light Mode — ThemeCubit 2.3

```
class ThemeCubit extends Cubit<ThemeState> {
  ThemeCubit() : super(ThemeState.initial()) {
    _loadTheme(); // ← theme يجب الـ
  }

  Future<void> setTheme(ThemeMode mode) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('theme_mode', mode == ThemeMode.dark ? 'dark' : 'light');
    emit(ThemeState(themeMode: mode));
  }

  void toggleTheme() {
    final newMode = state.themeMode == ThemeMode.dark
      ? ThemeMode.light
      : ThemeMode.dark;
    setTheme(newMode);
  }
}
```

- **Constructor:** يحمّل الـ `theme` المحفوظ تلقائياً عند الإنشاء.
- **setTheme:** يحفظ في `SharedPreferences` ثم `emit` حالة جديدة → `MaterialApp` يلوّن الـ `UI` بالـ `theme` الجديد.
- **toggleTheme:** يقلب بين `dark` و `light`. زرار الـ `ThemeToggleButton` ينادي عليها.

Realtime ال — WebSocketService 3.

Singleton Pattern 3.1

```
class WebSocketService {
    static WebSocketService? _instance;
    static WebSocketService get instance => _instance ??= WebSocketService._();
    WebSocketService._() { _initStreams(); }
}
```

ليه **Singleton**؟ مفيش معنى لفتح أكثر من WS connection. لما أي كود ينادي `WebSocketService.instance` يحصل على نفس النسخة في كل التطبيق.

`== ??` = "لو null اعمل assignment، غير كده سيبه". فال `instance` يتعمل مرة واحدة فقط.

3.2 ال Streams — قنوات البث

```
StreamController<Map<String, dynamic>>? _gradeUpdateController;

Stream<Map<String, dynamic>> get gradeUpdateStream {
    if (_gradeUpdateController == null || _gradeUpdateController!.isClosed) {
        _initStreams();
    }
    return _gradeUpdateController!.stream;
}
```

StreamController = مصدر لل `stream`. الكود اللي بيستقبل `messages` من السيرفر يستخدم `_gradeUpdateController.add(data)` ليثبت.

Stream = القناة اللي ال `al` يستمع لها. مثل اشتراك في قناة يوتيوب: كل ما تنزل حلقة (event) المشتركين يشوفوها.

broadcast() = يسمح بأكثر من listener (المعتاد `stream` له listener واحد فقط).

3.3 connect()

```

Future<void> connect() async {
  if (_isConnected && _channel != null) return;          // (1)

  final token = await SecureStorageService.getToken();   // (2)
  final userData = await SecureStorageService.getUserData();
  if (token == null || token.isEmpty || userData == null) return;

  final user = jsonDecode(userData);
  final userRole = user['role'] ?? 'guest';
  String clientType = userRole == 'doctor' ? 'mobile-doctor' : 'mobile-student';

  final wsUrl = '${AppConstants.wsUrl}?type=mobile';    // (3)

  final socket = await WebSocket.connect(wsUrl, customClient: httpClient)
    .timeout(const Duration(seconds: 12));              // (4)

  _channel = IOWebSocketChannel(socket);
  _isConnected = true;

  _channel!.stream.listen(                              // (5)
    (message) => _handleMessage(message),
    onDone: () { _isConnected = false; _scheduleReconnect(); },
    onError: (error) { _isConnected = false; _scheduleReconnect(); },
  );

  sendMessage({                                         // (6)
    'type': 'REGISTER',
    'clientType': clientType,
    'userId': user['id'],
  });

  _startHeartbeat();                                   // (7)
}

```

1. لو متصل بالفعل → ارجع. (تجنب اتصال مزدوج).
2. اقرأ الـ token والـ user من SecureStorage. لازم يكون فيه login عشان نتصل.
3. جهز الـ URL: `wss://traxa-system.online?type=mobile`.
4. اتصل مع 12 timeout ثانية.
5. استمع للرسائل الجاية من السيرفر. لو الاتصال انقطع → جدول reconnect.
6. أرسل أول رسالة REGISTER لتعريف نفسك للسيرفر.

7. ابدأ heartbeat (نبضات كل 25 ثانية للحفاظ على الاتصال).

3.4 handleMessage — توزيع الرسائل

```
void _handleMessage(dynamic message) {
  final data = jsonDecode(message);
  switch (data['type']) {
    case 'GRADE_UPDATED':
      _gradeUpdateController?.add(data);
      break;
    case 'SESSION_ACTIVATED':
      _sessionActivatedController?.add(data);
      break;
    case 'SEMESTER_CHANGED':
      _semesterController?.add(data['semester'] as int);
      break;
    case 'PING':
      sendMessage({'type': 'PONG'});
      break;
    ...
  }
}
```

كل نوع رسالة عنده stream خاص. ال widgets المستمعة بتأخذ فقط النوع اللي يهمها بدل ما تستمع لكل شيء. مثلاً شاشة الدرجات تستمع ل gradeUpdateStream فقط.

3.5 scheduleReconnect — Exponential Backoff

```
void _scheduleReconnect() {
  if (!_isConnected) return;
  final delay = Duration(seconds: min(5 * (_reconnectAttempts + 1), 30));
  _reconnectTimer = Timer(delay, () {
    _reconnectAttempts++;
    connect();
  });
}
```

الفكرة: لو الاتصال انقطع، استنى 5 ثواني وحاول. لو فشل تاني، 10 ثواني. ثم 15. حتى 30 ثانية كحد أقصى. ده اسمه "Exponential Backoff" — يقلل ضغط السيرفر لو فيه مشكلة كبيرة.

4. Repositories — وسيط بين ال Cubit وال API

4.1 BaseRepository.guard () — أهم دالة

```
Future<Result<T>> guard<T>(Future<T> Function() action, {String? tag}) async {
  try {
    return Success(await action());
  } on AppException catch (e) {
    return Failure(e);
  } on SocketException catch (e, s) {
    return Failure(NetworkException('No internet connection', cause: e, stack: s));
  } on http.ClientException catch (e, s) {
    return Failure(NetworkException(e.message, cause: e, stack: s));
  } on async.TimeoutException {
    return Failure(const TimeoutException());
  } catch (e, s) {
    return Failure(UnknownAppException(e.toString(), cause: e, stack: s));
  }
}
```

اللي بتعمله:

1. تشغل ال **action** (مثلاً HTTP call).
2. لو نجح: ترجع **Success(value)**.
3. لو نزل exception، تمسكه وتحوله لـ **Failure(typed exception)**.

ليه مهمة؟

- ال Cubit ما يحتاجش يكتب try/catch في كل دالة.
- الأخطاء بقت (NetworkException، TimeoutException...) typed فال UI يعرف يفرّق.
- ال `Result<T>` يفرض على الكود يعالج الحالتين (sealed class) بتعني المترجم هيشككي لو نسيت (Failure).

4.2 AuthRepository.studentLogin ()

```
Future<Result<({User user, String token})>> studentLogin({...}) async {
  return guard(() async {
```

```

final response = await ApiService.studentLogin(studentId, password, isFirstLogin);
final outcome = fromLegacyResponse(response);
return outcome.when(
  success: (data) async {
    final json = Map<String, dynamic>.from(data['student']);
    json['role'] = 'student';
    json['userType'] = 'student';
    json['username'] ??= json['student_id']?.toString();
    final user = User.fromJson(json);
    await _persistSession(user, data['token'] as String, fallbackType: 'student');
    return (user: user, token: data['token'] as String);
  },
  failure: (e) => throw e,
);
}, tag: _tag);
}

```

1. تشغيل login HTTP عبر ApiService.
2. `fromLegacyResponse` — تحول الـ `{... , success': true'}` القديم إلى `Result`.
3. تضيف role/userType (مش بيرجعوا من السيرفر للطلاب).
4. تنشئ User object.
5. تحفظ في SecureStorage (الـ session).
6. ترجع `(user: ..., token: ...)` `.tuple`.

Dart 3: `(user: u, token: t)` = object في **Record (Tuple)** صغير بـ named fields بدون الحاجة لتعريف class. الـ Cubit يستخدمه: `session.user`, `session.token`.

4.3 AttendanceRepository.waitForCameraStarted()

```

Future<bool> waitForCameraStarted({
  required Object sessionId,
  required String token,
  Duration timeout = const Duration(seconds: 90),
  Duration interval = const Duration(milliseconds: 500),
}) async {
  final deadline = DateTime.now().add(timeout);
  while (DateTime.now().isBefore(deadline)) {
    try {
      final res = await http.get(...).timeout(const Duration(seconds: 3));
    }
  }
}

```

```
    if (res.statusCode == 200) return true;
  } catch (_) {}
  await Future.delayed(interval);
}
return false;
}
```

:Polling Pattern

1. كل نص ثانية، اتصل بالسيرفر واسأل: "صفحة الكاميرا اتفتحت على الـ host؟"
2. لو رد 200 → نعم، ارجع true.
3. لو فضلت 90 ثانية بدون رد → ارجع false.

ليه 90 ثانية؟ لما السيرفر يطلب فتح الكاميرا، صفحة الـ host تحمّل face-api models من GitHub CDN، اللي ممكن تاخذ دقيقة على شبكة الجامعة البطيئة.

Services .5

5.1 SecureStorageService — التخزين المشفر

```
class SecureStorageService {
  static const _storage = FlutterSecureStorage(
    aOptions: AndroidOptions(encryptedSharedPreferences: true),
    iOptions: IOSOptions(accessibility: KeychainAccessibility.first_unlock),
  );

  static Future<void> saveToken(String token) =>
    _write(_tokenKey, token, 'Token');

  static Future<String?> getToken() =>
    _read(_tokenKey, 'Token');
}
```

ليه مش **SharedPreferences** عادي؟

SharedPreferences بتحفظ نص خام يقدر أي حد عنده وصول للجهاز يقرأه. **SecureStorage** بتستخدم:

- Android: EncryptedSharedPreferences (مشفر AES).
- iOS: Keychain (hardware-backed).

ده مهم للـ **JWT token** لأنه يمثل هوية المستخدم.

5.2 CacheService — Offline Support

```
static Future<void> saveAllData({students, doctors, subjects, lectures}) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString(_cacheStudentsKey, jsonEncode(students));
  ...
  await prefs.setString(_cacheTimestampKey, DateTime.now().toIso8601String());
}

static Future<Map<String, List?>> loadAllData({bool ignoreExpiry = false}) async {
  final timestampStr = prefs.getString(_cacheTimestampKey);
  if (!ignoreExpiry && timestampStr != null) {
    final timestamp = DateTime.parse(timestampStr);
```

```

    if (DateTime.now().difference(timestamp) > _cacheMaxAge) return null;
  }
  ...
}

```

- **الحفظ:** نخزن البيانات كـ JSON strings + timestamp.
- **القراءة العادية:** لو الـ cache أقدم من ساعة، ارجع null (DataCubit يجلب من السيرفر).
- **القراءة بـ `ignoreExpiry=true`:** ارجع البيانات حتى لو قديمة. يستخدم لما الـ network يفشل (أفضل من شاشة فاضية).

5.3 Token — TokenHolder قابل للمراقبة

```

class TokenHolder {
  final ValueNotifier<String?> _token = ValueNotifier<String?>(null);

  ValueListenable<String?> get listenable => _token;

  Future<String?> load() async {
    final saved = await SecureStorageService.getToken();
    _token.value = saved;
    return saved;
  }

  Map<String, String> get authHeader =>
    hasToken ? {'Authorization': 'Bearer ${_token.value!}'} : const {};
}

```

ValueNotifier أبسط من Cubit للحالات الـ **authHeader** getter single-value. يرجع الـ HTTP header جاهز للاستخدام: `http.get(url, headers: tokenHolder.authHeader)`.

6. شاشات (Screens) — كيف تُبنى

LoginScreen 6.1

ال State variables

```
final _formKey = GlobalKey<FormState>();
final _usernameController = TextEditingController();
final _passwordController = TextEditingController();
final _isStudentLogin = ValueNotifier<bool>(true);
final _obscurePassword = ValueNotifier<bool>(true);
final _isSubmitting = ValueNotifier<bool>(false);
```

- **GlobalKey<FormState>** — مرجع للـ Form. نستخدمه لـ **validate()**.
- **TextEditingController** — يربط TextField بـ Dart string قابل للقراءة/الكتابة.
- **ValueNotifier** — متغيرات صغيرة قابلة للمراقبة (toggle طالب/دكتور، إخفاء/إظهار password).

handleLogin()

```
Future<void> _handleLogin() async {
  if (!_formKey.currentState!.validate()) return;
  _isSubmitting.value = true;

  final authCubit = context.read<AuthCubit>();
  await authCubit.login(
    username: _usernameController.text.trim(),
    password: _passwordController.text,
    isStudent: _isStudentLogin.value,
  );
  if (mounted) _isSubmitting.value = false;
}
```

1. **formKey.currentState!.validate()** — يشغل كل validators على الـ TextFormField. لو في حقل فاضي/خطأ، ينزل رسالة الخطأ تحته.
2. **isSubmitting.value = true** — الزرار يبقى loading.

3. `<context.read<AuthCubit>()` — يجيب الـ `AuthCubit` من الـ `Provider`.
4. `await authCubit.login(...)` — نستنى الـ `login` يخلص.
5. `if (mounted)` — تأكد إن الـ `widget` لسه على الشاشة. لو الشاشة اتقفلت أثناء `await` ما نعملش `setState` على `widget` مش موجود.

dispose() — تنظيف الموارد

```
@override
void dispose() {
  _usernameController.dispose();
  _passwordController.dispose();
  _isStudentLogin.dispose();
  _obscurePassword.dispose();
  _isSubmitting.dispose();
  super.dispose();
}
```

قاعدة: أي `Controller/Notifier` تعمله في `initState()` / `ك field`، لازم تـ `dispose` في `dispose()`. نسيانهم `memory leak =`

6.2 DoctorScreen — الـ Shell متعدد التابز

IndexedStack vs PageView

```
static const List<Widget> _sections = [
  DoctorOverview(),
  DoctorSubjects(),
  DoctorAttendance(),
  DoctorReports(),
  DoctorProfile(),
];

// في build:
IndexedStack(index: currentTab, children: _sections);
```

IndexedStack يحتفظ بحالة كل `tab` (مثلاً `scroll position`) عند التنقل. بديله `PageView` لكن `PageView` يدمر الـ `widgets` خارج الشاشة. لو خرجت من `Attendance` ورجعت لازم تعيد الـ `session` — مش حلو.

اشتراك في WebSocket Streams

```

final List<StreamSubscription> _wsSubs = [];

@override
void initState() {
  super.initState();
  _wsSubs.add(
    WebSocketService.instance.semesterStream.listen((semester) {
      _semester.value = semester;
      context.read<DataCubit>().refreshForNewSemester();
    }),
  );
  // المزيد من الاشتراكات ...
}

@override
void dispose() {
  for (final sub in _wsSubs) {
    sub.cancel();
  }
  super.dispose();
}

```

قاعدة ال Streams: كل اشتراك (subscription) لازم نلغيه في dispose، وإلا ال stream يفضل يبعث events ل widget اتدمر → crash.

7. Widgets — الكتل اللي بنبني بيها

7.1 Container — أعمدة المباني

مثال من CustomCard

```
Container(
  margin: margin ?? const EdgeInsets.all(0),
  child: Material(
    color: Colors.transparent,
    child: InkWell(
      onTap: onTap,
      borderRadius: BorderRadius.circular(24.r),
      child: Ink(
        padding: padding ?? EdgeInsets.all(20.r),
        decoration: BoxDecoration(
          color: isDark ? const Color(0xFF1E293B) : Colors.white,
          borderRadius: BorderRadius.circular(24.r),
          border: Border.all(color: isDark ? ... : Colors.grey.shade200),
          boxShadow: isDark ? null : [
            BoxShadow(color: Colors.black.withValues(alpha: 0.05),
              blurRadius: 10.r, offset: Offset(0, 2.h)),
          ],
        ),
        child: child,
      ),
    ),
  ),
);
```

شرح طبقات الكارت:

1. **Container** الخارجي: مسئول عن ال margin (المسافة من حوالية).
2. **Material**: ضروري عشان InkWell يشتغل (ripple effect).
3. **InkWell**: يخلي الكارت قابل للضغط ويعمل تأثير الضغطة.
4. **Ink**: بدلاً من Container هنا، عشان لون ال ripple يظهر على الكارت.
5. **BoxDecoration**: اللون + ال radius + border + shadow.

6. **child**: المحتوى اللي المستخدم بيمررلنا (نص/صورة/أي حاجة).

مكونات BoxDecoration

الخاصية	الفائدة	مثال
color	لون خلفية صلب	<code>Colors.white</code>
gradient	لون متدرج	<code>LinearGradient(colors: [...])</code>
borderRadius	تدوير الكورنرات	<code>BorderRadius.circular(16)</code>
border	حد حواليه	<code>Border.all(color: Colors.grey)</code>
boxShadow	ظل	<code>[BoxShadow(blurRadius: 10)]</code>
image	صورة خلفية	<code>DecorationImage(image: ...)</code>
shape	شكل (دائرة)	<code>BoxShape.circle</code>

GradientButton 7.2 — زرار متدرج اللون

```
AnimatedContainer(
  duration: const Duration(milliseconds: 200),
  width: double.infinity,
  height: 56.h,
  decoration: BoxDecoration(
    gradient: LinearGradient(colors: colors),
    borderRadius: BorderRadius.circular(16.r),
    boxShadow: isLoading ? const [] : [
      BoxShadow(color: theme.primaryColor.withValues(alpha: 0.4),
        blurRadius: 12.r, offset: Offset(0, 4.h)),
    ],
  ),
  child: ElevatedButton(
    onPressed: isLoading ? null : onPressed,
    style: ElevatedButton.styleFrom(
      backgroundColor: Colors.transparent,
      shadowColor: Colors.transparent,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16.r)),
    ),
    child: isLoading
      ? CircularProgressIndicator(strokeWidth: 2.5, color: Colors.white)
  )
)
```

```
: Row(children: [Text(label), Icon(icon)]),
),
);
```

- **AnimatedContainer**: زي Container عادي لكن لما تتغير أي قيمة (لون، حجم...) يعمل animation سلس بدل ما يقفز فوراً.
- **gradient**: بنفسجي → indigo.
- **boxShadow**: ظل بنفسجي شفاف يدي إحساس "مرتفع".
- **ElevatedButton**: backgroundColor transparent عشان ال gradient من Container يظهر.
- **isLoading**: لو CircularProgressIndicator → true، غير كده النص + الأيقونة.
- **onPressed: isLoading ? null : onPressed**: تعطيل الزرار أثناء التحميل.

7.3 ProfileAvatar — صورة بروفایل مع cache

```
Container(
  width: size, height: size,
  decoration: BoxDecoration(shape: BoxShape.circle, color: backgroundColor),
  child: ClipOval(
    child: CachedNetworkImage(
      imageUrl: url,
      fit: BoxFit.cover,
      placeholder: (_, __) => _initial(), // أول حرف ←
      errorWidget: (_, __, ___) => _initial(), // لو الصورة فشلت ←
    ),
  ),
);
```

- **ClipOval**: يقص أي محتوى لشكل بيضاوي/دائري.
- **CachedNetworkImage**: يحمل الصورة من URL + يخزنها في cache (مش يحمل تاني).
- **placeholder**: يظهر أثناء التحميل.
- **errorWidget**: يظهر لو الصورة مش موجودة (أول حرف من الاسم).

evict() static method

```
static Future<void> evict(String url) async {
  await CachedNetworkImage.evictFromCache(url);
}
```



```
await CachedNetworkImageProvider(url).evict();
PaintingBinding.instance.imageCache.clear();
PaintingBinding.instance.imageCache.clearLiveImages();
}
```

لما ال User يحدث صورته، ال URL هو نفسه لكن الصورة اتغيرت. لازم نمسح ال cache من 3 طبقات عشان الصورة الجديدة تظهر فوراً.

AppSkeleton — Loading Shimmer 7.4

```
Skeletonizer(
  enabled: enabled,
  enableSwitchAnimation: true,
  effect: ShimmerEffect(
    baseColor: isDark ? Color(0xFF1E293B) : Colors.grey.shade300,
    highlightColor: isDark ? Color(0xFF334155) : Colors.grey.shade100,
    duration: const Duration(milliseconds: 1100),
  ),
  child: child,
);
```

الفكرة: تلف أي widget بـ `Skeletonizer(enabled: isLoading, child: widget)`. لما `isLoading=true`، Flutter يستبدل النصوص والأشكال بـ shimmer رمادي يتحرك. مفيش حاجة تحتاج تعديل.

WsConnectionDot 7.5 — نقطة الحالة

```
StreamBuilder<bool>(
  stream: Stream.periodic(
    const Duration(seconds: 1),
    (_) => WebSocketService.instance.isConnected,
  ),
  builder: (context, snapshot) {
    final color = snapshot.data ?? false ? Colors.green : Colors.red;
    return Container(
      width: 8.w, height: 8.w,
      decoration: BoxDecoration(shape: BoxShape.circle, color: color,
        boxShadow: [BoxShadow(color: color.withValues(alpha: 0.5), blurRadius: 4.r)],
      ),
    );
  });
```

```
},  
) ;
```

- **Stream.periodic**: stream يبعث event كل ثانية. القيمة المُبعَثَة = حالة WS الحالية.
- **StreamBuilder**: يستمع ويستعيد البناء كل event.
- **النتيجة**: نقطة خضراء/حمراء ب glow تتحدث كل ثانية.

البديل الأفضل: WebSocketService يستخدم ValueNotifier و Listen عليه بدل polling. لكن للبساطة، هنا polling يكفي (مرة كل ثانية مش مكلف).

8. حسابات ال helpers.dart — GPA

8.1 calculateGPA()

```
double calculateGPA(List<Grade> grades, List<Subject> subjects) {
  double totalPoints = 0;
  int totalCredits = 0;
  for (final grade in grades) {
    if (!grade.isVisible) continue;
    final subject = findSubjectSafely(grade.subjectId, subjects);
    if (subject == null) continue;
    final credits = subject.totalCreditHours;
    totalPoints += grade.gradePoints * credits;
    totalCredits += credits;
  }
  return totalCredits > 0 ? (totalPoints / totalCredits) : 0.0;
}
```

المعادلة:

$$\text{GPA} = \frac{\sum (\text{gradePoints} \times \text{creditHours})}{\sum \text{creditHours}}$$

1. تخطّ الدرجات غير المرئية (admin قفلها).
2. لكل grade، جيب ال subject عشان نعرف الكريدتس.
3. اضرب ال gradePoints (4.0 لـ A+، 0.0 لـ F) في الكريدتس.
4. اقسم المجموع على مجموع الكريدتس.

gradePoints (في model Grade)

```
double get gradePoints {
  if (total >= 96) return 4.0;
  if (total >= 92) return 3.7;
  if (total >= 88) return 3.4;
  ...
  if (total >= 50) return 1.0;
  return 0.0;
}
```

ال letter له قيمة عددية بين **Scale:** 12-point scale (A+/A/A-/B+/B/B-/C+/C/C-/D+/D/D-/F) كل letter له قيمة عددية بين 0.0 و 4.0.

8.2 calculateEarnedCredits()

```
int calculateEarnedCredits(List<Grade> grades, List<Subject> subjects) {  
    int total = 0;  
    for (final grade in grades) {  
        if (grade.total >= 50 && grade.isVisible) {  
            final subject = findSubjectSafely(grade.subjectId, subjects);  
            if (subject != null) total += subject.totalCreditHours;  
        }  
    }  
    return total;  
}
```

كم ساعة معتمدة نجح فيها الطالب ($\text{drrgs} \geq 50$).

9. توليد pdf_report_service.dart — PDF

9.1 generateAndOpen()

```
final pdf = pw.Document();

pdf.addPage(pw.MultiPage(
  pageFormat: PdfPageFormat.a4,
  margin: const pw.EdgeInsets.all(40),
  header: (ctx) => _header(...),
  footer: (ctx) => pw.Row(...), // page X of Y
  build: (ctx) => [
    pw.SizedBox(height: 10),
    _summary(...),
    pw.SizedBox(height: 20),
    pw.Text('Student Details', ...),
    _studentsTable(students),
  ],
));

final dir = await getApplicationDocumentsDirectory();
final file = File('${dir.path}/${fileName}');
await file.writeAsBytes(await pdf.save());

await OpenFile.open(file.path);
```

1. **pw.Document()** — وثيقة PDF فاضية.
2. **pw.MultiPage** — صفحات متعددة تلقائياً.
3. **header/footer** — يتكرروا على كل صفحة.
4. **build** — يرجع list من widgets؛ الـ MultiPage يقسمها على الصفحات.
5. **pdf.save()** — يحول الـ Document لـ bytes.
6. **writeAsBytes** — يحفظ على القرص.
7. **OpenFile.open** — يفتح الـ PDF بالـ app الافتراضية.

pw. = widgets خاصة بالـ PDF (مش flutter widgets). API مشابه لكن منفصل.

9.2 studentsTable_ () — جدول الطلاب

```
final rows = <pw.TableRow>[
  pw.TableRow(
    decoration: pw.BoxDecoration(color: PdfColor.fromHex('#8B5CF6')),
    children: headers.map((h) => pw.Container(
      padding: const pw.EdgeInsets.symmetric(horizontal: 4, vertical: 6),
      child: pw.Text(h, style: pw.TextStyle(fontWeight: pw.FontWeight.bold, color: PdfColors.w
    )),
  )),
];

for (var i = 0; i < students.length; i++) {
  final s = students[i];
  rows.add(pw.TableRow(
    decoration: pw.BoxDecoration(color: i % 2 == 0 ? PdfColors.grey100 : PdfColors.white),
    children: [
      cell('${i + 1}'),
      cell(s['student_id_number'] ?? ''),
      cell(s['student_name'] ?? 'Unknown'),
      // Status box ملون
      pw.Container(
        color: statusBg,
        child: pw.Text(statusLabel, style: pw.TextStyle(color: PdfColors.white)),
      ),
      cell(hasFace ? 'Yes' : 'No', color: hasFace ? green : red),
      cell(hasQr ? 'Yes' : 'No', color: hasQr ? green : red),
      cell(duration),
    ],
  ));
}

return pw.Table(border: ..., columnWidths: widths, children: rows);
```

- **Header row:** لون بنفسجي + نص أبيض **bold**.
- **Zebra rows:** `i % 2 == 0 ? grey100 : white` — صفوف بلون متناوب لقراءة أسهل.
- **Status colored box:** أخضر (Present)، أصفر (Pending)، أحمر (Absent).
- **Face/QR Yes/No:** لون النص يبين النتيجة.
- **Duration:** الفرق بين وقت ال Face detect ووقت QR scan.

10. مفاهيم متقدمة شائعة في الكود

10.1 ال ?? و ?.

المعامل	المعنى	مثال
??	"لو null استخدم الافتراضي"	'name ?? 'Guest
=??	"لو null اعمل assignment"	()cache ??= getData
?.	"لو null ما تنزلش"	user?.name
!	"أنا متأكد إنه مش null"	user!.name

10.2 copyWith pattern

```
User copyWith({String? name, int? level}) {
  return User(
    name: name ?? this.name,
    level: level ?? this.level,
    ...
  );
}

// استخدام:
final updated = user.copyWith(name: 'Ahmed');
```

ال Models immutable (ال final fields). لو عايز تغير حقل واحد، تنشئ نسخة جديدة بكل الحقول + التغيير. ده يحمي من تعديلات عرضية ويتوافق مع BLoC (ال Equatable يقدر يقارن بسهولة).

10.3 Cascade Operator

```
final list = []
  ..add('a')
  ..add('b')
  ..add('c');
// = list.add('a'); list.add('b'); list.add('c');
```

... Spread Operator 10.4

```
final allItems = [...firstList, ...secondList, lastItem];
```

Pattern Matching (Dart 3) 10.5

```
result.when(  
  success: (value) => print('OK: $value'),  
  failure: (error) => print('Error: $error'),  
);  
  
// أو switch expression:  
final color = switch (status) {  
  'confirmed' => Colors.green,  
  'pending'   => Colors.yellow,  
  _           => Colors.red,  
};
```


11. أنماط مهمة (Patterns) في المشروع

Repository Pattern 11.1

ال Cubits لا تتحدث مباشرة مع HTTP أو storage. كل حاجة تمر عبر Repository. الفائدة:

- سهل اختبار ال Cubits بـ FakeRepository.
- لو غيرنا source البيانات (مثلاً GraphQL بدل REST)، نغير repository فقط.

Service Locator (GetIt) 11.2

```
getIt.registerLazySingleton<AuthRepository>(() => AuthRepository());

// الاستخدام:
final repo = getIt<AuthRepository>();
```

بدل ما تمرر AuthRepository من main لكل widget جوه، أي كود يقدر يجيبها من **GetIt. lazySingleton** يتعمل أول مرة بس يتم طلبه (مش وقت ال register).

Result<T> Pattern 11.3

```
sealed class Result<T> {}
class Success<T> extends Result<T> { final T value; }
class Failure<T> extends Result<T> { final AppException exception; }
```

بدل ما الدوال ترمي exceptions، ترجع Result. الكود اللي بيستدعيها مضطر يعالج الحالتين (success و failure) لأن ال sealed بتجبره.

Singleton with lazy initialization 11.4

```
static WebSocketService? _instance;
static WebSocketService get instance => _instance ??= WebSocketService._();
```

Local State لـ Listenable / ValueNotifier 11.5

بدل Cubit للحالات الصغيرة جداً (boolean toggle مثلاً)، نستخدم ValueNotifier + ValueListenableBuilder. أخف وأسرع.

12. نصائح للمذاكرة

1. ابدأ بـ **main.dart** — افهم تدفق التشغيل من `ServerConfig` → `ServiceLocator` → `AuthGate`.
2. افتح **AuthCubit + AuthState** — افهم كيف الـ `states` تتغير وكيف الـ `UI` تتفاعل.
3. اقرأ **BaseRepository.guard()** — أكثر دالة استخداماً في المشروع كله.
4. افتح أي شاشة (مثلاً **DoctorOverview**) — لاحظ كيف تستخدم `BlocBuilder` تجيب الـ `data` وتعرضها.
5. **WebSocketService** — افهم الـ `singleton + streams + reconnect`. ده الجزء "السحري" اللي بيعمل `.realtime`.
6. **helpers.dart** — حسابات GPA مفيدة لو سؤال عنها في المناقشة.
7. **pdf_report_service.dart** — مهم لو الـ `committee` سأل عن إخراج التقارير.

تمرين: جرب تتبع رحلة "طالب يفتح التطبيق ويسجل دخول":

```
main → ServerConfig.load → initServiceLocator → AuthCubit.init → LoginScreen →
_handleLogin → AuthCubit.login → AuthRepository.studentLogin → ApiService.studentLogin
→ HTTP POST → response → SecureStorage.save → emit Success → AuthGate
→ StudentScreen يظهر → Streams → WebSocketService.connect → DataCubit.fullReload تستقبل
.events
```

— نهاية الوثيقة —